

Projeto e Análise de Algoritmos I

Aula 10 - Busca em Grafos (BFS e DFS)

Lucas Nunes Alegre

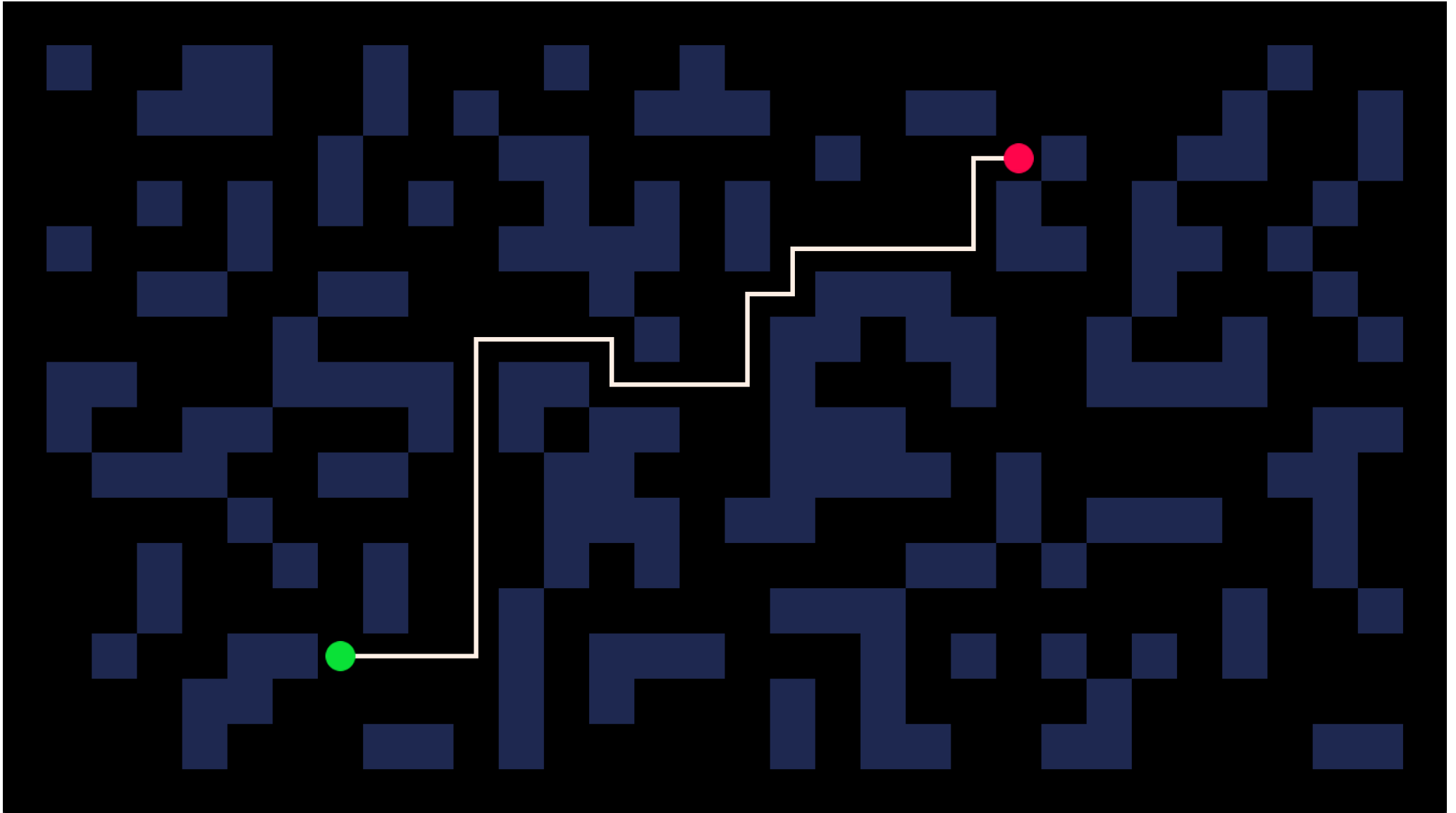
Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Teórica



Estes slides utilizam conteúdo adaptado da bibliografia da disciplina e também de notas de aula e slides prévios dos professores Bruno Grisci, Rodrigo Machado, André Grahl Pereira, Lucas Nunes Alegre, Marcus Ritt e Luciana Buriol.

Busca em Grafos

Pathfinding



Pathfinding



Algoritmos de Busca em Grafos

Problemas de **busca em grafos**:

Conectividade: Dado um Grafo G e um nó v , descobrir todos os nós alcançáveis a partir de v .

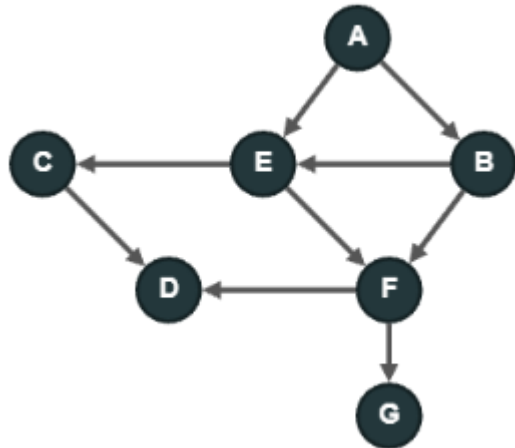
Caminho Mais Curto: Dado um Grafo G e um nó v , descobrir o caminho mais curto a partir de v para todos os outros nós.

Existem duas estratégias principais para algoritmos de busca em grafos:

- busca em profundidade / **depth-first search (DFS)**
- busca em largura / **breadth-first search (BFS)**

Buscas em Grafos (Conceitualmente)

$G1 =$



1. Buscar em largura (a partir de B)

2. Buscar em profundidade (a partir de B)

Pergunta: em que **ordem** os vértices serão alcançados em cada uma das estratégias?

Pergunta: como podemos avaliar o **custo** desses procedimentos?

Custos em Grafos

Um grafo simples $G = (V, E)$ possui dois parâmetros naturais:

- $|V| = n$ (número de vértices)
- $|E| = m$ (número de arestas)

Esses parâmetros (n e m) fazem parte da análise, e dependendo pode não ser muito claro como se relacionam.

Por exemplo, qual seria o melhor custo assintótico: $O(m^2)$ ou $O(n^2)$?

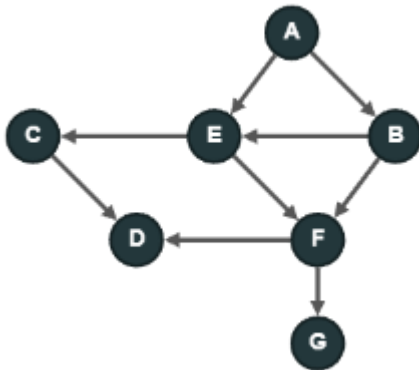
Observe que há algumas relações já vistas entre n e m :

- $m \leq \binom{n}{2} \leq n^2$ (em grafos simples)
- $m \geq n - 1$ (em grafos conexos)

Custo das Representações de Grafos e Dígrafos

Matriz de adjacência

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>	<i>G</i>
<i>A</i>	0	1	0	0	1	0	0
<i>B</i>	0	0	0	0	1	1	0
<i>C</i>	0	0	0	1	0	0	0
<i>D</i>	0	0	0	0	0	0	0
<i>E</i>	0	0	1	0	0	1	0
<i>F</i>	0	0	0	1	0	0	1
<i>G</i>	0	0	0	0	0	0	0



Lista de adjacência (Python)

```
1 ## Grafo como dicionario
2 g1 = { 'A' : ['B', 'E'],
3       'B' : ['E', 'F'],
4       'C' : ['D'],
5       'D' : [],
6       'E' : ['C', 'F'],
7       'F' : ['D', 'G'],
8       'G' : [] }
```

- Como $m \leq n^2$, o custo $O(m + n)$ nunca é pior que $O(n^2)$, sendo muito melhor para grafos *esparsos*.
- Listas são mais eficientes para implementar **buscas** pelo grafo.

Complexidade: Matriz vs. Lista de Adjacência

Seja $n = |V|$ e $m = |E|$.

Operação	Matriz	Lista
Espaço	$O(n^2)$	$O(n + m)$
Verificar aresta (u, v)	$O(1)$	$O(\deg(u))$
Listar vizinhos de v	$O(n)$	$O(\deg(v))$
Inserir aresta	$O(1)$	$O(1)$
Remover aresta	$O(1)$	$O(\deg(u))$

Algoritmo Genérico de Busca em Grafos

Algoritmo Genérico de Busca

- Dado um grafo $G = (V, E)$ e um vértice fonte s
- Marca s como **explorado** e todos os outros vértices como **não explorados**
- Enquanto for possível:
 - Escolhe uma aresta $e = (u, v)$, onde u já foi explorado e v ainda não foi explorado.
 - Marca v como explorado.

Algoritmo Genérico de Busca

BUSCA-GENÉRICA($G : \text{Grafo}, s : \text{Nodo}$)

- 1: Marca s como explorado
 - 2: **while** existir vértice v não explorado **do**
 - 3: Escolhe aresta (u, v) onde u foi explorado e v ainda não foi explorado
 - 4: Marca v como explorado
-

Correção do Algoritmo Genérico

Afirmção: Ao fim do **Algoritmo Genérico**, um vértice v é **marcado como explorado**, se e somente se, **existe um caminho** entre s e v .

Prova ($\Rightarrow$): Assuma que o vértice v é marcado explorado. Faremos indução no número de iterações.

- **Caso base:** $P(0) = s$ explorado, e s tem um caminho para si mesmo.
- **Hipótese indutiva:** Assuma $P(n)$, ou seja, todos os nodos explorados até a iteração n possuem caminho a partir de s .
- **Faremos $P(n) \rightarrow P(n + 1)$:**
 - Na iteração $n + 1$, um nodo v é explorado via aresta (u, v) , onde u já estava explorado.
 - Pela hipótese, u tem caminho a partir de s .
 - Concatenando o caminho $s \rightarrow u$ com (u, v) , obtemos um caminho de s para v .

Correção do Algoritmo Genérico (Cont.)

Afirmção: Ao fim do **Algoritmo Genérico**, um vértice v é **marcado como explorado**, se e somente se, **existe um caminho** entre s e v .

Prova ($\Rightarrow$): Assuma que existe um caminho entre s e v .

- Por **contradição**, assumimos que v **não termina marcado como explorado**.
- Como s foi explorado logo de cara e o caminho termina em v (não explorado), deve existir alguma aresta $e = (a, b)$ no caminho $s \rightarrow v$ na qual a é explorado e b não é explorado.
- Mas se há aresta e obedecendo esse critério, o laço de busca do **Algoritmo Genérico** poderia tê-la escolhido, o que implica que o algoritmo ainda não teria terminado!
- **Contradição.** ■

Busca em Profundidade (DFS)

Visualização

Visualização: BFS vs. DFS - https://www.youtube.com/shorts/1-elk8F8_UM

BFS

BFS vs. DFS

Busca em Profundidade (Iterativa)

Busca em profundidade (DFS) pode ser implementada de forma **iterativa**:

- Utilizamos uma **pilha** para registrar nodos a serem visitados
- Marcamos nodos como **visitados após a inserção na pilha**
- A pilha inicia com o nodo inicial da busca
- O laço principal roda **enquanto a pilha não é vazia**
- Dentro do corpo do laço, removemos o nodo da pilha e o processamos
- Para cada vizinho não visitado do nodo atual, fazemos a sua **inserção na pilha e o marcamos como visitado**

Busca em Profundidade (DFS)

DFS(G, s)

- 1: Marca s como visitado
 - 2: $S \leftarrow [s]$ // Pilha (LIFO)
 - 3: **while** S não vazia **do**
 - 4: $v \leftarrow S.\text{pop}()$
 - 5: **for** cada $(v, w) \in E$ **do**
 - 6: **if** w não visitado **then**
 - 7: Marca w como visitado
 - 8: Adiciona w a S
-

Busca em Profundidade (Iterativa) em Python

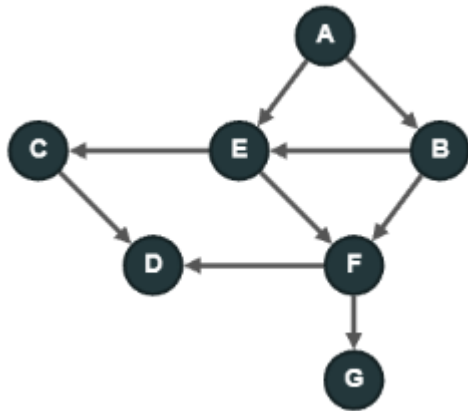
Python Code

↺ Start Over

▶ Run Code

```
1 v g1 = { 'A' : ['B', 'E'],
2         'B' : ['E', 'F'],
3         'C' : ['D'],
4         'D' : [],
5         'E' : ['C', 'F'],
6         'F' : ['D', 'G'],
7         'G' : [] }
8
9 ## Busca em profundidade (DFS, iterativo)
10 v def dfs_iter(g, s): # g e um grafo, s e um nodo
11     visited = {i : False for i in g.keys()}
12
13     p = [s]; # inicia pilha p com s
14     visited[s] = True # registra que s foi visitado
15
16 v while len(p) > 0: # enquanto ha elementos em p
17     h = p.pop(0) # extrai cabeca de p em h
18     print(h) # faca algo com h
19 v for i in g[h]: # para vizinhos de h
20 v     if not visited[i]: # se vizinho nao visitado
21         p.insert(0, i) # insere na cabeca da pilha
22         visited[i] = True # marca como visitado
23
24 dfs_iter(g1, "A")
```


Busca em Profundidade (Iterativa): Exemplo



Resultado de `dfs_iter(g1, 'A')`:

A E F G D C B

```
1 ## Grafo como dicionario
2 g1 = { 'A' : ['B', 'E'],
3       'B' : ['E', 'F'],
4       'C' : ['D'],
5       'D' : [],
6       'E' : ['C', 'F'],
7       'F' : ['D', 'G'],
8       'G' : [] }
```

Busca em Profundidade (Recursiva)

Busca em profundidade (DFS) pode ser implementada de forma bem simples por meio de **recursão**:

- Utilizamos uma **vetor global** para registrar nodos que já foram visitados, inicializado com false
- A rotina principal recebe um grafo e um nó de partida
- Após processar o nó, marca ele como visitado
- Para cada vizinho não visitado do nó atual, fazemos uma chamada recursiva para explorá-lo

Busca em Profundidade (Recursiva) em Python

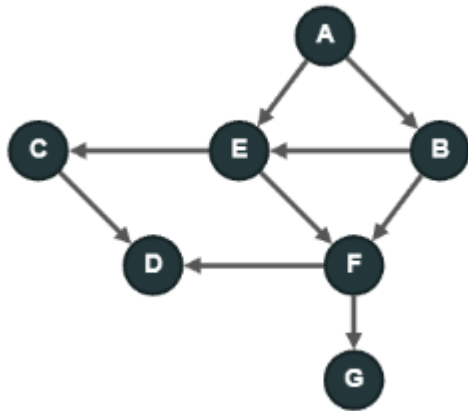
Python Code

↺ Start Over

▶ Run Code

```
1 v g1 = { 'A' : ['B', 'E'],
2         'B' : ['E', 'F'],
3         'C' : ['D'],
4         'D' : [],
5         'E' : ['C', 'F'],
6         'F' : ['D', 'G'],
7         'G' : [] }
8
9 v def init_visitados(g):
10     """Dicionário mapeando todo nodo para False (não-visitado)."""
11     return {n : False for n in g.keys()}
12
13 v def dfs_rec(g, s, visitados):
14     """Busca em profundidade a partir de um nodo s (versão recursiva)."""
15     visitados[s] = True
16     print(s)
17 v     for v in g[s]:
18 v         if not visitados[v]:
19             dfs_rec(g, v, visitados)
20
21 ## Chamada de teste (DFS recursivo)
22 dfs_rec(g1, 'A', init_visitados(g1))
```

Busca em Profundidade (Recursiva): Exemplo



Resultado de `dfs(g1, 'A')`:

A B E C D F G

Pergunta: Você consegue justificar por que os nodos são visitados em ordem distinta da versão iterativa?

```
1 ## Grafo como dicionario
2 g1 = { 'A' : ['B', 'E'],
3       'B' : ['E', 'F'],
4       'C' : ['D'],
5       'D' : [],
6       'E' : ['C', 'F'],
7       'F' : ['D', 'G'],
8       'G' : [] }
```

Busca em Largura (BFS)

Busca em Largura (Níveis)

Breadth-First Search (BFS): Explora a partir de s todos os vértices adicionando vértices em camadas.

BFS (Camadas):

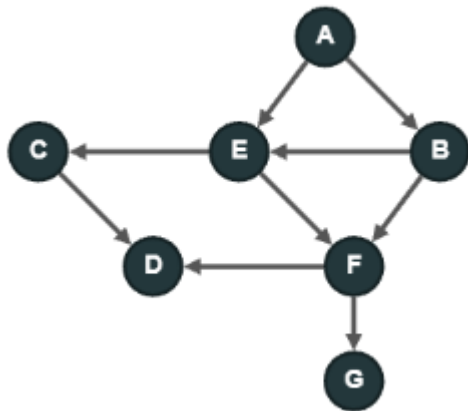
- $L_0 = \{s\}$
- $L_1 =$ todos vértices adjacentes a s
- $L_2 =$ todos vértices que não pertencem a L_0 ou L_1 e que possuem uma aresta que conecta a um vértice em L_1
- $L_{i+1} =$ todos vértices que não pertencem a camadas anteriores e que possuem uma aresta que conecta a um vértice em L_i

Busca em Largura (Níveis)

- Busca em largura pode ser pensada como uma busca em **níveis**.
- Um nodo no n -ésimo nível está há uma distância de n arestas da origem.

No grafo de exemplo, temos os seguintes níveis (a partir de 'A'):

- **0**: A
- **1**: B, E
- **2**: C, F
- **3**: D, G



Busca em Largura (Iterativa)

- BFS também pode ser implementado de uma forma bastante simples utilizando uma **fila**.
- O código é **essencialmente idêntico à versão iterativa de DFS**, só mudando a **estrutura de dados**:
 - **Pilha / LIFO** (inserção na frente, remoção na frente): DFS
 - **Fila / FIFO** (inserção atrás, remoção na frente): BFS

Busca em Largura (BFS)

BFS(G, s)

- 1: Marca s como explorado
 - 2: $Q \leftarrow [s]$ // Fila (FIFO)
 - 3: **while** Q não vazio **do**
 - 4: $v \leftarrow Q.pop()$
 - 5: **for** cada $(v, w) \in E$ **do**
 - 6: **if** w não explorado **then**
 - 7: Marca w como explorado
 - 8: Adiciona w a Q
-

Busca em Largura (Iterativa) em Python

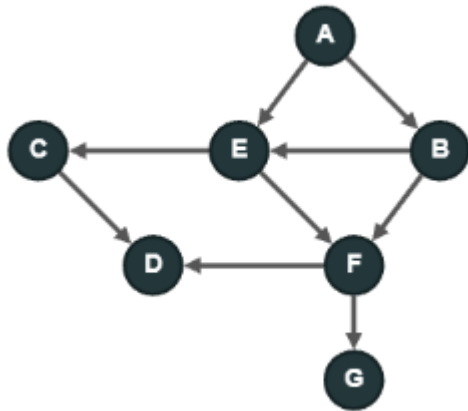
Python Code

↺ Start Over

▶ Run Code

```
1  ## Busca em largura (BFS, iterativo)
2  from collections import deque
3
4  g1 = { 'A' : ['B', 'E'],
5         'B' : ['E', 'F'],
6         'C' : ['D'],
7         'D' : [],
8         'E' : ['C', 'F'],
9         'F' : ['D', 'G'],
10        'G' : [] }
11
12  def bfs_iter(g, s): # g um grafo, s um nodo
13      visited = {i : False for i in g.keys()}
14
15      queue = deque([s])    # inicia lista unitaria com s
16      visited[s] = True     # registra que s foi visitado
17
18      while len(queue) > 0: # enquanto ha elementos em l
19          h = queue.popleft() # extrai cabeca da lista em h
20          print(h)           # faca algo com h
21          for i in g[h]:     # para vizinhos de h
22              if not visited[i]: # se vizinho nao visitado
23                  queue.append(i) # insere no final da lista
24                  visited[i] = True # marca como visitado
```


Busca em Largura (Iterativa): Exemplo



Resultado de `bfs_iter(g1, 'A')`:

A B E F C D G

```
1 ## Grafo como dicionario
2 g1 = { 'A' : ['B', 'E'],
3       'B' : ['E', 'F'],
4       'C' : ['D'],
5       'D' : [],
6       'E' : ['C', 'F'],
7       'F' : ['D', 'G'],
8       'G' : [] }
```

Custo de Operações em Python: dict

Operação	Custo
<code>k in d</code>	$O(1)$ em média, $O(n)$ no pior caso
<code>d[k]</code>	$O(1)$ em média, $O(n)$ no pior caso
<code>d[k] = v</code>	$O(1)$ em média, $O(n)$ no pior caso
<code>del d[k]</code>	$O(1)$ em média, $O(n)$ no pior caso
Iteração / cópia	$O(n)$

Fonte: <https://wiki.python.org/moin/TimeComplexity>

Custo de Operações em Python: list

Operação	Custo
<code>l[i], l[i] = x, len(l)</code>	$O(1)$
<code>append(x)</code> , <code>pop()</code> no fim	$O(1)$ amortizado
<code>x in l</code> , iteração	$O(n)$
<code>insert(i, x)</code> , <code>pop(i)</code> , <code>del l[i]</code>	$O(n)$
<code>l[i:j]</code>	$O(k)$

Fonte: <https://wiki.python.org/moin/TimeComplexity>

Custo de Operações em Python:

`collections.deque`

Operação	Custo
<code>append(x)</code> , <code>appendleft(x)</code>	$O(1)$
<code>pop()</code> , <code>popleft()</code>	$O(1)$
<code>len(d)</code>	$O(1)$
<code>extend(t)</code> , <code>extendleft(t)</code> , <code>rotate(k)</code>	$O(k)$
<code>remove(x)</code>	$O(n)$

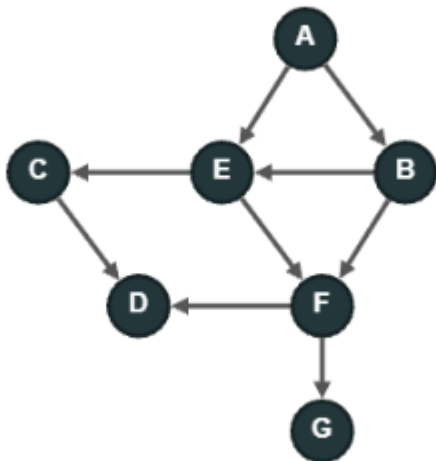
Fonte: <https://wiki.python.org/moin/TimeComplexity>

Árvore Associada à Busca

Árvore Associada à Busca

Árvore de Busca. Dada uma busca (DFS ou BFS) em G a partir de um nó v , construir uma **árvore** associada a essa busca onde **cada nó aponta para o elemento que o alcançou** diretamente.

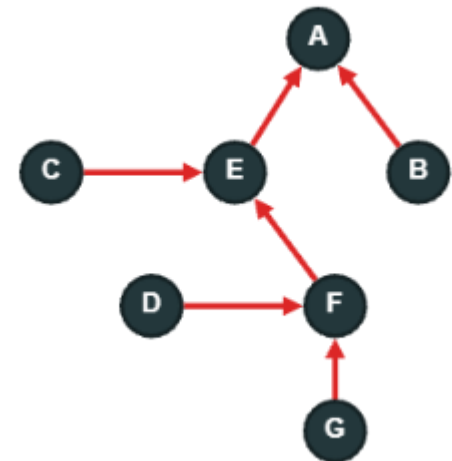
Dígrafo g_1 :



`dfs_iter(g1,
'A'):`

```
1 A  
2 E  
3 F  
4 G  
5 D  
6 C  
7 B
```

Árvore de busca:



- Isso é facilmente implementado com um dicionário auxiliar `parent`, preenchido ao inserirmos um nó na estrutura.
- A raiz da árvore é o nó de origem.

Arvore de busca (DFS iterativa)

Veja a modificação do algoritmo DFS iterativo para construir e retornar a árvore:

Python Code

↺ Start Over

▶ Run Code

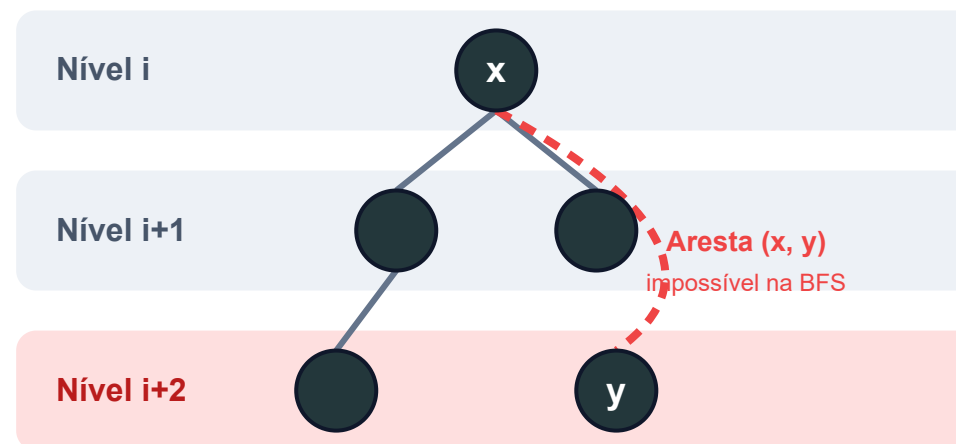
```
1  from collections import deque
2
3  def dfs_iter_tree(g, v):
4      """Árvore de Busca de DFS. Retorna dicionário mapeando cada nodo a seu nodo pai na busca.
5      parent = {i : i for i in g.keys()} # arvore de antecessor
6      visited = {i : False for i in g.keys()}
7      p = [v]
8      visited[v] = True
9
10     while len(p) > 0:
11         h = p.pop(0)
12         for i in g[h]:
13             if not visited[i]:
14                 p.insert(0,i)
15                 parent[i] = h # marca h como antecessor de i
16                 visited[i] = True
17
18     return parent # devolve a arvore
19
20  g1 = { 'A' : ['B', 'E'], 'B' : ['E', 'F'], 'C' : ['D'], 'D' : [],
21         'E' : ['C', 'F'], 'F' : ['D', 'G'], 'G' : [] }
22  print(dfs_iter_tree(g1, 'A'))
```


Propriedades da Busca em Largura (BFS)

Propriedade dos Níveis: Seja T a árvore resultante da BFS para o grafo $G = (V, E)$ e (x, y) uma aresta qualquer de G :

- Então, os níveis de x e y na árvore de busca diferem **no máximo por 1**.

Isto ocorre porque, se a diferença fosse maior (ex: y dois ou mais níveis abaixo de x), no momento em que a BFS visitou os vizinhos de x , o vértice y teria sido colocado na fila logo no nível abaixo de x , limitando a diferença de nível a no máximo 1.



Contradição: y deveria estar no nível i+1!

Complexidade Assintótica das Buscas em Grafos

Complexidade da Busca em Largura (BFS)

Afirmção: O algoritmo BFS tem tempo de execução $O(n + m)$.

Detalhes da Prova:

Complexidade da Busca em Largura (BFS)

BFS(G, s)

- 1: Marca s como explorado // $O(1)$
 - 2: $Q \leftarrow [s]$ // Fila (FIFO) - $O(1)$
 - 3: **while** Q não vazio **do** // Laço executa n vezes no total
 - 4: $v \leftarrow Q.pop()$ // $O(1)$
 - 5: **for** cada vizinho w de v **do** // Itera $deg(v)$ vezes por nó, totalizando $O(m)$
 - 6: **if** w não explorado **then** // $O(1)$
 - 7: Marca w como explorado // $O(1)$
 - 8: Adiciona w a Q // $O(1)$
-

Análise Assintótica das Buscas em Grafos

- As versões iterativas de BFS e DFS utilizando filas e pilhas (respectivamente) possuem custo assintótico $O(n + m)$.
- Abaixo usaremos *estrutura* para mencionar a pilha ou fila utilizada:

Detalhamento:

- Há um processo de iniciação do vetor/dicionário de visitados $O(n)$
- A estrutura é inicializada com um elemento inicial, e o laço principal finaliza com a estrutura vazia
- A cada passada pelo corpo do laço principal, removemos um elemento da estrutura e, ao final, adicionamos elementos
- O número de elementos adicionados à estrutura corresponde ao número de arestas dos vértices percorridos na busca: no pior caso, $O(m)$

Pergunta Conceitual

Considere um grafo com **quantidade infinita de nós** (mas **nenhum nó com grau infinito**).

P. Qual a diferença de comportamento da DFS e BFS nesse caso?

Implementação: Python

Ver Github da disciplina: https://github.com/BrunoGrisci/projeto-e-analise-de-algoritmos/blob/main/paa1/graph_search.ipynb

